

MiNiMax前端AI开发

一面（技术基础） · 1-6

1. 请简述 SSE（Server-Sent Events）与 WebSocket 的区别，为什么 AI 对话场景多采用 SSE？
2. 在实现 AI 聊天列表时，如何处理长文本渲染导致的页面卡顿？
3. 请手写一个 Fetch 处理 ReadableStream 的基本逻辑，模拟获取 AI 流式输出。
4. 如何实现一个自动滚动到底部的聊天容器，并处理用户手动向上滚动时的冲突？
5. Promise.all 和 Promise.allSettled 的区别是什么？在调用多个模型接口时你会选哪个？
6. CSS 如何实现一个类似 ChatGPT 的打字机光标闪烁效果？

二面（深入原理与项目） · 1-8

1. 在 React 项目中，AI 逐字输出会导致组件频繁重渲染，你有哪些优化手段？
2. 如何设计一个可扩展的 Prompt 模板管理组件？
3. 前端如何参与 RAG（检索增强生成）流程的展示优化？
4. 谈谈 Web Worker 在 AI 前端应用中的实际使用场景。
5. 如何实现一个支持“多轮对话中断”和“重新生成”的状态机？
6. React Fiber 架构是如何提升 AI 对话这种高频交互场景的响应性的？
7. 手写一个处理流式数据并支持中途取消的自定义 Hook（useStream）。
8. 移动端 AI 聊天场景中，如何解决软键盘弹出遮挡输入框的问题？

三面（架构与综合能力） 1-6

1. 如果让你从零设计一个 AI Agent 的工作流可视化编辑器，你会如何选型和设计架构？
2. 如何建立一套针对前端 AI 应用的质量监控体系？
3. 在多模态（图片、语音、视频）交互中，前端架构如何保持灵活性？
4. 谈谈你对端侧模型（如 WebLLM）的看法，它在前端有哪些应用前景？
5. 如何处理 AI 幻觉（Hallucination）在 UI 层的反馈设计？
6. 聊聊你在过去一年中遇到的最难的 AI 前端技术挑战及解决方案。

一面（技术基础）

1. 请简述 SSE (Server-Sent Events) 与 WebSocket 的区别，为什么 AI 对话场景多采用 SSE？

难度：★★

考点：网络协议、流式传输

答案要点：

SSE 是一种基于 HTTP 的单向推送技术，相比 WebSocket 更轻量，天然支持断线重连且对防火墙更友好。

- 协议层面：SSE 基于标准 HTTP 协议，WebSocket 是独立的 TCP 协议。
- 数据流向：SSE 仅支持服务端向客户端推送，WebSocket 支持全双工双向通信。
- 场景适配：AI 输出是典型的单向流式过程，SSE 实现简单，支持自动重连，且不需要处理复杂的握手逻辑。
- 兼容性：SSE 在现代浏览器中原生支持，且能通过 Polyfill 兼容旧版，对代理服务器支持更好。

常见坑：

- 误认为 SSE 只能传输文本，其实可以通过 Base64 传输二进制。
- 忽略了 HTTP/1.1 下 SSE 有 6 个并发连接限制的问题。

追问：

- 如果需要客户端在流式输出过程中实时中断请求，你会怎么做？

2. 在实现 AI 聊天列表时，如何处理长文本渲染导致的页面卡顿？

难度：★★

考点：性能优化、虚拟列表

答案要点：

处理长文本渲染主要通过减少 DOM 节点数量和优化渲染频率来实现，核心手段是虚拟滚动和分段渲染。

- 虚拟滚动：只渲染可视区域内的对话气泡，通过监听滚动事件动态计算偏移量。
- 异步渲染：利用 requestAnimationFrame 或分片处理，避免大段 Markdown 一次性解析阻塞主线程。
- 样式优化：避免在对话容器中使用过多的 CSS 复杂选择器，减少重排重绘。
- 内存管理：及时销毁不可见区域的复杂组件实例，防止内存溢出。

常见坑：

- 虚拟列表在高度动态变化（AI 逐字输出）时，滚动条抖动问题。
- Markdown 渲染器（如 marked）解析过大文本时的同步阻塞。

追问：

- 如果对话中有大量公式（MathJax）或代码高亮，如何进一步优化？

3. 请手写一个 Fetch 处理 ReadableStream 的基本逻辑，模拟获取 AI 流式输出。

难度：★★★

考点：原生 API、流处理

答案要点：

通过 Fetch 获取 Response 对象后，利用 `body.getReader()` 循环读取数据块，并使用 `TextDecoder` 进行解码。

- 获取 Reader：调用 `response.body.getReader()` 获取可读流读取器。
- 循环读取：使用 `while(true)` 配合 `await reader.read()` 持续获取 value 和 done 状态。
- 文本解码：使用 `TextDecoder` 实例将 `Uint8Array` 转换为字符串。
- 状态处理：正确处理流关闭、网络异常及业务层面的错误码。

常见坑：

- 忘记处理 `TextDecoder` 跨 chunk 截断中文字符的问题（需保持 decoder 实例）。
- 没有处理用户主动关闭页面时的流释放。

代码块

```
1  async function getStream(url) {
2    const response = await fetch(url);
3    const reader = response.body.getReader();
4    const decoder = new TextDecoder();
5    while (true) {
6      const { done, value } = await reader.read();
7      if (done) break;
8      const text = decoder.decode(value, { stream: true });
9      console.log("收到片段:", text);
10   }
11 }
```

4. 如何实现一个自动滚动到底部的聊天容器，并处理用户手动向上滚动时的冲突？

难度：★★

考点：DOM 操作、用户体验

答案要点：

核心逻辑是判断当前滚动位置是否在底部，如果是则随新内容自动滚动，否则保持不动。

- 阈值判断：通过 `scrollHeight - scrollTop === clientHeight` 判断是否触底。
- 自动滚动：在内容更新后，使用 `scrollIntoView` 或设置 `scrollTop` 实现平滑滚动。
- 手动干预：监听 `wheel` 或 `touchmove` 事件，若用户向上滚动，则标记“禁止自动滚动”。
- 恢复机制：提供一个“回到最新消息”的按钮，点击后恢复自动滚动逻辑。

常见坑：

- 图片加载或公式渲染导致高度异步变化，滚动位置计算不准。

- 频繁触发滚动导致的性能抖动。

追问：

- 如何在 React/Vue 框架中优雅地封装这个逻辑？

5. Promise.all 和 Promise.allSettled 的区别是什么？在调用多个模型接口时你会选哪个？

难度：★

考点：异步编程

答案要点：

Promise.all 具有原子性，任何一个失败则全部失败；allSettled 会等待所有结果返回，无论成功或失败。

- 失败处理：all 遇到第一个 reject 立即返回，allSettled 记录每个 promise 的状态和值。
- 返回值结构：all 返回结果数组，allSettled 返回包含 status 和 value/reason 的对象数组。
- AI 场景选择：通常选 allSettled，因为即使某个模型接口挂了，其他模型的结果依然可以展示给用户。

常见坑：

- 在 all 中忘记捕获单个 promise 的错误，导致整个链路崩溃。

追问：

- 如果来实现“最快返回的一个结果”，应该用哪个 API？

6. CSS 如何实现一个类似 ChatGPT 的打字机光标闪烁效果？

难度：★

考点：CSS 动画

答案要点：

利用伪元素模拟光标，并通过 keyframes 动画控制 opacity 或 visibility 实现闪烁。

- 伪元素：在内容容器后添加 ::after，设置宽度、背景色和 display: inline-block。
- 动画定义：定义 0% 和 50% 状态的透明度切换。
- 步进函数：使用 steps(1) 或 linear 确保闪烁节奏。
- 动态控制：通过 CSS 类名控制光标在输出结束时消失。

常见坑：

- 光标导致行高变化，引起文字抖动。
- 在多行文本末尾定位不准。

代码块

```
1 .cursor::after {  
2   content: "";
```

```
3     display: inline-block;
4     width: 2px;
5     height: 1em;
6     background: #000;
7     animation: blink 1s steps(1) infinite;
8 }
9 @keyframes blink {
10     50% { opacity: 0; }
11 }
```

二面（深入原理与项目）

1. 在 React 项目中，AI 逐字输出会导致组件频繁重渲染，你有哪些优化手段？

难度：★★★

考点：渲染优化、状态管理

答案要点：

优化核心在于减少 UI 树的 Diff 范围和降低状态更新频率。

- 局部状态下放：将流式文本状态限制在最末端的 Text 组件中，避免父组件全量重绘。
- 缓存机制：使用 useMemo 缓存已生成的 Markdown 静态片段，只渲染新增部分。
- 节流更新：不要每收到一个字符就 setState，可以按时间片段（如 50ms）批量更新。
- 引用稳定：确保渲染 Context 或 Provider 的 value 引用稳定，防止无关子组件刷新。

常见坑：

- 在 Context 中存储流式文本导致整棵树重渲染。
- 频繁更新导致的输入框失去焦点（Focus）问题。

追问：

- 如果使用了 Redux，你会如何处理这种高频更新的数据？

2. 如何设计一个可扩展的 Prompt 模板管理组件？

难度：★★

考点：组件设计、抽象能力

答案要点：

采用配置驱动的设计模式，支持变量插槽、类型校验和版本控制。

- 模板解析：支持 {{variable}} 语法，动态提取变量生成表单输入项。
- 类型定义：支持文本、下拉、开关等多种输入类型，通过 JSON Schema 描述。
- 预览功能：实时渲染填充变量后的最终 Prompt，方便用户调试。

- 组合能力：支持 System Prompt 和 User Prompt 的解耦与组合。

常见坑：

- 忽略了变量转义问题，导致 Prompt 注入风险。
- 模板过长时的交互体验不佳。

3. 前端如何参与 RAG (检索增强生成) 流程的展示优化？

难度：★★★

考点：AI 业务理解、交互设计

答案要点：

前端主要负责检索来源的展示、引用高亮以及用户反馈闭环。

- 引用标注：在 AI 回复中通过 [1] 形式标注来源，点击可跳转或悬浮展示原文片段。
- 来源溯源：展示检索到的知识库文档列表，支持 PDF/Docx 预览及关键词高亮。
- 评分反馈：提供点赞/踩功能，并允许用户修正错误的检索结果，回传给后端微调。
- 进度感知：展示“正在检索知识库...”、“正在总结...”等中间状态，缓解焦虑。

常见坑：

- 引用索引与后端返回数据匹配不上的逻辑错误。
- 预览超大文档时的前端性能问题。

4. 谈谈 Web Worker 在 AI 前端应用中的实际使用场景。

难度：★★

考点：多线程、计算密集型任务

答案要点：

Web Worker 主要用于处理不阻塞主线程的计算任务，如本地模型运行、重文本解析或向量计算。

- 本地 Embedding：在前端进行简单的文本向量化计算（如使用 Transformers.js）。
- 语法高亮解析：将复杂的 Markdown/Code 解析逻辑移入 Worker。
- 大文件预处理：用户上传知识库文档时，在 Worker 中进行分段（Chunking）和清洗。
- 离线模型：运行轻量级的端侧 LLM（如 WebLLM 框架）。

常见坑：

- Worker 与主线程频繁通信带来的序列化开销。
- 无法直接操作 DOM 的限制。

5. 如何实现一个支持“多轮对话中断”和“重新生成”的状态机？

难度：★★★

考点：逻辑设计、状态机

答案要点：

通过定义明确的状态 (Idle, Generating, Paused, Error) 和 Action 来管理复杂的对话流。

- 中止逻辑：利用 AbortController 关联 Fetch 请求，在用户点击“停止”时触发 abort()。
- 覆盖更新：重新生成时，需定位到当前消息 ID，清空后续内容并重新触发流式请求。
- 历史回溯：支持修改历史提问，此时需要截断后续对话树，发起新的分支请求。
- 幂等性：确保多次点击重新生成不会导致状态错乱。

常见坑：

- 请求已 abort 但前端状态未重置，导致 UI 卡死。
- 重新生成时未处理好 Token 消耗的统计。

6. React Fiber 架构是如何提升 AI 对话这种高频交互场景的响应性的？

难度：★★★

考点：React 原理

答案要点：

Fiber 通过时间分片和优先级调度，确保高优先级的用户输入不被低优先级的长文本渲染阻塞。

- 时间分片：将渲染工作拆分为小片，在浏览器空闲时执行，避免主线程长时间占用。
- 优先级调度：用户在输入框打字的优先级高于 AI 文本的流式渲染。
- 可中断性：当有更高优先级的任务进来时，Fiber 可以暂停当前的 Diff 过程。
- 并发模式：利用 useTransition 或 useDeferredValue 进一步优化流式数据的展示。

常见坑：

- 误以为 Fiber 能自动解决所有性能问题，忽略了组件本身的优化。

7. 手写一个处理流式数据并支持中途取消的自定义 Hook (useStream)。

难度：★★★

考点：React Hooks、流处理

答案要点：

封装 AbortController 和流读取逻辑，暴露数据、加载状态和取消方法。

- 状态管理：维护 data, loading, error 等状态。
- 引用控制：使用 useRef 存储 AbortController 实例。
- 清理机制：在 useEffect 的 return 中调用 abort，防止组件卸载后继续更新。
- 回调支持：提供 onFinish, onError 等钩子。

代码块

```
1 function useStream() {
2   const [data, setData] = useState("");
3   const abortCtrl = useRef<AbortController | null>(null);
```

```
4
5  const fetchStream = async (url: string) => {
6    abortCtrl.current = new AbortController();
7    const res = await fetch(url, { signal: abortCtrl.current.signal });
8    const reader = res.body?.getReader();
9    // ... 读取逻辑 ...
10   setData(prev => prev + chunk);
11 };
12
13 const stop = () => abortCtrl.current?.abort();
14
15 return { data, fetchStream, stop };
16 }
```

8. 移动端 AI 聊天场景中，如何解决软键盘弹出遮挡输入框的问题？

难度：★★

考点：移动端适配、H5 坑点

答案要点：

这是一个经典的 H5 兼容性问题，需结合 VisualViewport API 和布局调整来解决。

- VisualViewport：监听 window.visualViewport.onresize，动态计算键盘高度。
- 布局调整：将输入框容器设为 fixed 或 absolute，根据键盘高度动态修改 bottom 值。
- 滚动修正：在输入框聚焦时，调用 scrollToView({ block: 'end' }) 确保光标可见。
- 避坑指南：iOS 上键盘弹出不会触发 window.resize，需监听 focusin 事件。

常见坑：

- iOS 12+ 键盘收起后页面不回弹导致的点击位移。
- 某些安卓浏览器键盘弹出直接压缩整个 WebView 导致布局错乱。

三面（架构与综合能力）

1. 如果让你从零设计一个 AI Agent 的工作流可视化编辑器，你会如何选型和设计架构？

难度：★★★★

考点：系统设计、技术选型

答案要点：

架构设计应侧重于图形引擎的选择、数据模型的抽象以及插件化扩展能力。

- 引擎选型：选择 React Flow 或 X6 作为基础绘图库，因其具备良好的节点自定义能力和事件系统。
- 数据协议：定义标准的 JSON DAG (有向无环图) 结构，包含 Nodes, Edges 和 Data 属性。

- 状态管理：使用 Zustand 或 Redux 管理全局画布状态，支持撤销重做（Undo/Redo）。
- 节点抽象：设计通用的 BaseNode，支持 LLM 节点、API 调用节点、条件判断节点等扩展。
- 调试能力：支持单步执行、节点状态实时回显（如输出日志、Token 消耗）。

常见坑：

- 节点过多时的 Canvas/SVG 性能瓶颈。
- 循环引用检测逻辑缺失。

2. 如何建立一套针对前端 AI 应用的质量监控体系？

难度：★★★

考点：监控、工程化

答案要点：

除了常规的性能指标，需重点关注 AI 特有的业务指标和异常捕获。

- 性能指标：首字响应时间 (TTFT)、平均每秒生成字数 (TPS)、流式中断率。
- 异常监控：SSE 连接断开频率、Markdown 解析失败率、模型接口 5xx 占比。
- 业务指标：用户点踩率、Prompt 长度分布、对话轮数统计。
- 链路追踪：将前端 TraceID 与后端日志关联，方便排查流式输出卡顿问题。

常见坑：

- 忽略了流式请求中途断开的监控，只监控了请求起始。
- 监控数据量过大导致的性能损耗。

3. 在多模态（图片、语音、视频）交互中，前端架构如何保持灵活性？

难度：★★★

考点：架构设计、扩展性

答案要点：

通过组件协议化和多媒体处理流水线来解耦不同模态的逻辑。

- 消息协议：定义统一的消息对象格式，通过 type 字段（text, image, audio, file）分发渲染组件。
- 渲染工厂：实现 MessageRenderer 工厂类，根据消息类型动态加载对应的展示组件。
- 输入扩展：设计统一的 InputBar，支持插件式挂载语音录制、图片上传、文件解析等模块。
- 预处理流水线：建立前端处理链，如图片压缩、语音转码（WebAudio API）、视频抽帧。

常见坑：

- 各种模态代码耦合严重，导致 InputBar 组件逻辑爆炸。

4. 谈谈你对端侧模型（如 WebLLM）的看法，它在前端有哪些应用前景？

难度：★★

考点：技术视野、前瞻性

答案要点：

端侧模型是隐私保护和降低成本的重要趋势，虽然目前受限于算力和显存，但潜力巨大。

- 优势：极低的延迟（无需网络）、数据不出本地（隐私安全）、节省服务端算力成本。
- 应用场景：本地文档摘要、敏感信息脱敏、简单的代码补全、离线语音助手。
- 技术瓶颈：浏览器 WebGPU 支持尚不普及、模型权重下载体积大（GB 级别）、显存占用高。
- 混合架构：未来可能是“云端大模型 + 端侧小模型”的协同模式。

常见坑：

- 盲目追求端侧运行，忽略了用户设备的硬件差异。

5. 如何处理 AI 幻觉（Hallucination）在 UI 层的反馈设计？

难度：★★

考点：产品思维、交互设计

答案要点：

前端应通过交互手段引导用户质疑和验证 AI 的输出。

- 确定性提示：对置信度低的内容进行视觉弱化或标注“可能存在偏差”。
- 事实核查：提供“搜索验证”按钮，一键调用搜索引擎对比 AI 结果。
- 引用溯源：强要求 AI 输出必须附带知识库来源，前端提供便捷的对比视图。
- 交互引导：在用户复制内容时提示“请注意核实信息准确性”。

常见坑：

- UI 设计过于信任 AI，导致用户被误导后产生投诉。

6. 聊聊你在过去一年中遇到的最难的 AI 前端技术挑战及解决方案。

难度：★★★

考点：项目经验、解决问题能力

答案要点：

（此题为开放题，需结合实际项目回答，以下为示例方向）

- 挑战示例：如何在低配安卓机上流畅渲染万字以上的流式 Markdown 对话？
- 解决思路：引入了基于时间片的增量解析器，结合虚拟列表和 Canvas 局部刷新。
- 结果：首屏加载提升 50%，滑动帧率从 20fps 提升至 55fps。
- 总结：通过对渲染链路的深度拆解，找到了瓶颈在于 DOM 节点过多导致的样式计算。

常见坑：

- 描述过于笼统，没有具体的数据支撑。
- 方案缺乏对比，没说明为什么选择 A 而不是 B。